

Web 自动探索开源代码调研

横纵分析法深度研究报告

作者: Codex

调研日期：2026-09-11 | 重点：AWorld、Go-Browse、Crawljax、Explorer、browser-use、Crawl4AI

结论与阅读顺序

如果目标是“给一个网站入口，让系统自己探索页面与功能，并留下以后能用的网站知识”，本次最值得优先研究的是 **Go-Browse** 的探索调度、**Crawljax** 的状态图，以及 **AWorld / Recon-Act** 的经验工具化。**browser-use** 适合作为浏览器执行层，**Crawl4AI** 适合批量发现链接与提取内容，**Explorer** 适合合成操作轨迹。这些项目不是同一个层面的替代品。

建议按这个顺序阅读：Go-Browse → Crawljax → AWorld 的 VisualWebArena 示例 → browser-use。若目标仅是抓取文档站内容，可以直接从 Crawl4AI 开始。

这是源码与文档调研，未安装运行这些浏览器系统，未复现论文分数。AWorld 的目录核验基于 GitHub API 返回的提交 `b4d24300d9f24e46d1018531a4afc67df8c43005`；其他项目读取的是调研时可访问的主分支代码。下文把源码事实、论文描述和工程建议分开，不把论文架构自动视为全部开源。

1. 先把“自动探索”变成可检查的能力

以一个购物网站为例，三种系统可能都说自己能自动浏览，但交付完全不同。

第一种系统从首页提取链接，访问商品页，输出商品名称和价格。它回答“网站有哪些内容”，主要处理 URL 和正文。

第二种系统收到“找到最便宜的红色背包”，主动搜索、筛选、排序，然后返回商品。它回答“这个任务怎么完成”，主要处理当前任务的行动序列。

第三种系统事先不知道最终任务。它尝试分类、筛选、排序、商品详情和购物车入口，记录哪些操作会改变页面、怎样回到已知位置，以及哪些能力已经被验证。它回答“这个网站能做什么、怎么到达、有什么前提”。这才是本报告重点讨论的站点探索。

一个可复用的探索器至少要处理六件事：

能力	具体问题	可检查的产物
页面观察	当前有哪些可操作元素？	DOM、可访问性树、截图
候选发现	下一步还有哪些入口没试？	待探索队列
状态判等	这是新页面，还是旧页面的新状态？	状态标识与归并规则
路径恢复	离开以后怎样回到这里？	到达路径与重放结果
预算控制	探索多久、何时停止重复？	步数、调用数、停止原因
知识沉淀	哪些结论可供后续任务使用？	带证据的网站能力说明

“保存轨迹”不等于“形成网站知识”。轨迹是某次执行发生过什么；网站知识需要进一步概括页面、对象、关系和操作前提；可复用 workflow 则描述以后如何完成类似任务。本报告沿用这个区分，不据此声称已经检查过你本地项目的现状。

2. 技术路线如何走到今天

2.1 从链接遍历走向交互状态遍历

只按链接爬取时，最自然的节点是 URL。当前页面有哪些链接，就把未访问的链接加入队列。广度优先先访问近邻，深度优先沿某条路径深入，按分数优先则把更相关的链接提前处理。Crawl4AI 的深度抓取将这些策略做成可配置组件。[深度抓取文档](#)

现代网页让这个表示不再充分。点击“高级筛选”可能只打开一个浮层，地址完全不变；购物车内容会变化，但路径仍是同一个；相反，跟踪参数变化可能产生新地址，却没有带来新的功能。于是“发现了多少 URL”和“理解了多少网站状态”开始分离。

Crawljax 所代表的路线是实际触发网页事件，把交互后的页面状态加入图，再继续尝试候选事件。它的价值在于明确管理状态和转换，而非依靠语言模型临时记住哪些按钮点过。[Crawljax 仓库](#)

2.2 语言模型带来语义选择，但没有自动解决覆盖问题

browser-use 的执行循环能够观察页面、请求模型选择动作、执行动作，再记录结果。模型使“选择一个筛选项”之类的语义判断更容易表达；页面解析和浏览器动作也不再要求每个网站单独写一套脚本。[Agent 主循环](#)

但一个任务完成得很好，不代表探索覆盖全面。模型可能一直围绕搜索栏工作，根本没打开账户菜单。把提示词改成“探索整个网站”可以得到一条浏览轨迹，却不能单独保证有全局待探索队列、稳定去重和断点恢复。这些仍是独立的工程问题。

2.3 探索的产物开始分化：训练数据、站点图、工具

2025 年的 Explorer 与 Go-Browse 代表探索用于收集训练轨迹的路线。Explorer 从网页出发生成并调整任务；Go-Browse 进一步将页面探索组织成外层图与内层任务采样。它们都让数据收集不必完全依赖人工事先编写任务，但输出仍主要服务训练。Explorer、Go-Browse

Recon-Act 则关注执行中的经验如何变成提示或专门工具。它对比失败与成功轨迹，提炼有用辅助能力，再交给执行团队使用。论文在 2025-09-25 提交，描述的实现达到 Level 3，仍保留人工参与，不能理解成完整无人值守的自进化系统。Recon-Act 论文

这几条路线可以组合，但没有一条天然等于完整的 Context Model 生成器。训练数据强调示范质量，爬虫强调访问与提取，工具库强调可执行能力，站点知识还要补上证据、适用范围和有效性管理。

3. AWorld：最值得看的是 Recon-Act 的公开边界

3.1 仓库中有三条不同的浏览器相关线

入口	实际定位	阅读目的
<code>examples/browser_use/</code>	集成到 AWorld 的 browser-use 衍生实现	浏览器 agent 如何接入框架
<code>examples/visualwebarena/</code>	Recon-Act Action Team 推理示例	网站经验如何辅助动作决策
<code>aworld-skills/agent-browser/</code>	浏览器 CLI 的操作说明与辅助内容	观察、点击、状态保存等工具接口
<code>examples/browser_use/README.md</code>	明确说明其实现源自 browser-use 并进行了框架适配；这条线并非独立的全站探索算法。固定版本说明	
<code>agent-browser/SKILL.md</code>	提供导航、快照、元素引用、点击、填写和会话状态保存等使用方法。存在浏览器操作技能，并不证明系统会自动为每个新网站生成业务知识。固定版本技能文件	

3.2 论文机制与开源示例要分开看

Recon-Act 论文提出侦察团队与行动团队协作。侦察团队分析轨迹、生成帮助执行的工具；行动团队分解意图、选择工具和生成动作。但公开示例 README 明确把自己定位为 Action Team 的推理代码。本次没有在这一示例目录里核验到可独立启动的完整侦察、分析、生成、验证流水线。[示例 README](#)

更具体地说，论文的 Level 3 配置仍让 Analyst 和 Tool Manager 保留人工参与。因此“论文提出自进化架构”“仓库提供工具化执行示例”和“所有探索步骤均已自动开源”是三个不同结论。本次只能确认前两项。[论文实现层级](#)

3.3 代码如何把知识变成行动

先读 `action_team.py` 的 `ActionTeam.next_action()`。它让 Master 根据任务、历史和工具描述选择辅助工具；先调用 hint 工具，将提示交给 Execution Agent；再调用 decision 工具，若返回了决策结果，就用它作为最终响应。

再读 `tool_manager.py`。它从 `hint_tools/` 和 `decision_tools/` 读取 Python 文件中的 `desc`，通过动态导入实例化同名类，并调用 `run()`。这是轻量工具注册与分发的具体实现，不是自动生成工具的代码。

最直观的例子是 `CategoryGuide.py`：

1. 检查当前 URL 是否以特定购物站地址开头。
2. 读取同目录的 `shopping_categories.json`。
3. 请模型从已有分类中选择目标分类。
4. 根据任务中的价格排序意图拼接参数。
5. 返回直接导航到目标 URL 的动作。

它说明一种很实用的经验表达方式：将“分类名称对应哪个地址、排序参数是什么”保存成结构化数据和少量代码，执行时减少重复探索。但这个文件本身没有展示分类表怎样从未知网站自动发现。

3.4 对后续方案的启发

AWorld 最适合借鉴的是 探索经验的消费方式。如果已经通过探索发现商品分类与排序规则，可以选择把它们写成网站知识，或者生成类似 `CategoryGuide` 的工具。

两者的区别在于：知识让执行模型自行理解并选择动作；工具提前封装了部分决策和执行逻辑。比较效果时应分别测试，否则不知道收益来自知识补充，还是更强的工具直接跳过了操作步骤。

对新网站迁移时还要处理已核验示例里的站点地址绑定和数据表依赖。可以复用模式，不能假设复制文件后就能自动适配所有购物网站。

4. Go-Browse：最接近可借鉴的探索主干

4.1 两层循环解决两个问题

Go-Browse 外层维护已发现页面的图，选择下一页；内层在该页提出功能任务和导航任务，尝试执行、筛选可行任务，再采集轨迹。这样每次探索能接着已有发现深入，不必每次都从首页重新猜。[项目说明](#)

对网站知识生成而言，最有价值的部分不是最后的微调脚本，而是前面的页面发现、到达路径保存、任务提出和验证机制。

4.2 已核验的源码入口

文件	看什么
<code>webexp/explore/algorithms/web_explore.py</code>	外层探索、任务提出与可行性检查
<code>webexp/explore/core/graph.py</code>	已探索/待探索节点、去重和恢复
<code>webexp/explore/core/node.py</code>	单个页面的任务、描述、路径与持久化
<code>configs/go_browse_config.yaml</code>	节点上限、模型、步数和环境参数

在 `web_explore.py` 中，`sample_task_candidates_for_node()` 收集探索器提出的任务；
`filter_to_feasible_tasks_for_node()` 尝试求解并保留成功信息；
`sample_task_solving_trajectories_for_node()` 进一步采样。
`process_open_urls_callback()` 则关联新发现的页面与到达它的轨迹前缀。

“轨迹前缀”就是到达某页面所需的前面一段操作。例如首页→打开菜单→进入订单列表。把它留下，后续探索可以复用已走通的路线，而不必再次让模型从零推理。

4.3 源码比架构图更朴素

`graph.py` 有两个决定迁移效果的细节：`nodes` 按 URL 字符串索引；`get_next_node()` 直接取待探索列表的第一个节点。它不是按语义信息增益排序，也没有在这一层按 DOM 状态判等。

这带来两个具体风险：同 URL 下的筛选浮层会被视为同一个节点；只有跟踪参数不同的地址又可能成为不同节点。若要做单页应用探索，需要增加更合适的状态表示，不能把“已有图”理解成“已解决状态爆炸”。

同一文件的 URL 规则也需理解清楚：先匹配 allow，再匹配 deny，剩余默认允许。因此这里的 allowlist 不是严格的只允许列表。接入限定站点探索时，应明确修改或包裹其匹配语义，而不是只填一个域名就认为范围已经严格限制。

4.4 持久产物已经很接近探索证据库

node.py 会保存 node_info.json，包括 URL、描述、children、visited 等信息，同时组织 prefixes/、tasks/、exploration_tasks/。图层保存 graph_info.json，提供 Graph.load() 恢复。

这些内容可作为网站知识的输入，但还需要做语义整理。例如“这个节点下面采到了三个成功任务”并不等于已经形成了“订单可按状态筛选，筛选条件会影响导出范围”这种具有明确前提和证据的知识。

4.5 起跑方式与改造边界

仓库要求安装 BrowserGym、浏览器和项目依赖，并配置目标环境与模型服务。现存配置文件是 configs/go_browse_config.yaml，README 示例命令使用了另一个配置文件名；阅读和运行时应以实际文件为准。[配置文件](#)

准备好依赖并填完配置后，可从该仓库根目录尝试：

```
python -m webexp.explore.algorithms.web_explore \
-c configs/go_browse_config.yaml
```

此命令是基于代码入口整理的启动方式，本次没有执行。配置支持开放网站入口，但这不代表任意网站的登录、状态重置和验证器都能直接复用。若目标是生成 Context Model，可先保留探索与证据层，暂停训练数据扩增和微调部分，避免一开始背上完整研究流水线。

5. Crawljax：理解“同一个网页的不同状态”

Crawljax 是更传统的交互式网页爬取路线，使用 Java/Selenium。它将浏览器交互产生的页面状态组成状态转换图，对动态网页而言，比只记录 URL 更贴近真实交互结构。[仓库](#)

建议优先读三个入口：

- StateVertexImpl.java：状态如何表示与比较。
- InMemoryStateFlowGraph.java：节点和转换如何组织。
- OutputBuilder.java：探索结果如何生成报告。

默认 `StateVertexImpl` 的 `equals()` 和 `hashCode()` 基于 `strippedDom`，也就是经处理的 DOM 表示。它还能查询候选元素是否存在未探索动作。这样，“URL 没变但 DOM 变了”有机会被识别为新状态。

但 DOM 比较也有局限。时间、广告、随机 ID 等变化可能制造伪新状态；过度剥离又可能掩盖真正的业务差异。因此需要根据网站配置归一化和状态抽象。本次核验的是默认状态类，不把整个项目所有扩展算法都归为同一种比较方式。

报告插件的产物包括 `result.json`、`config.json`、`alchemyGraph.json`、HTML 报告，以及 DOM 和截图文件。它提供的是可检查的结构证据，业务含义仍需进一步提炼。

我的判断是：如果已有 Python 浏览器执行层，先借鉴它的状态表示、候选动作管理和转换记录，不必立即再接入一套 Java 浏览器运行时。如果决定直接采用 Crawljax，则需要另行验证目标站点上的交互覆盖、表单输入和状态恢复效果。

6. Explorer：适合生成轨迹，不宜当成全站地图

Explorer 的目标是探索驱动的多模态网页轨迹合成。它从起始页面生成任务，在交互过程中调整目标，然后总结并验证轨迹。它与“用户给定一个任务、agent 只负责完成”的模式不同，但也不等于维护完整的网站探索队列。

关键入口是 `traj_gen/main.py`。本次源码核验看到任务生成、逐步 refinement、summarization 和 verification；产物包含 `task_trajectory_data.json`、各步 HTML、截图和带元素标记的截图，以及步骤日志。

在已检查的主流程中，没有核验到类似 Go-Browse 的跨 episode 站点 frontier 和状态图去重。因此更适合参考“如何从页面提出有意义的任务、如何整理并验证交互轨迹”。若直接用于网站知识构建，需要在它上面加全局探索调度。

官方的启动方式以 `python -m traj_gen.main` 为入口，需配置模型目录、起始 URL 和步数等参数。完整运行环境与模型开销未在本次验证，不把它列为开箱即用的轻量爬虫。

7. browser-use 与 Crawl4AI：两个基础组件

browser-use：观察与执行

在 `browser_use/agent/service.py` 中，`step()` 串起上下文准备、模型动作生成、动作执行和后处理。历史记录和回调可作为采集探索证据的接入点。

DOM service 则处理页面 DOM、可访问性信息与序列化表示，包含 `get_dom_tree()` 和 `get_serialized_dom_tree()` 等入口。可访问性树是以按钮、输入框、名称等信息描述页面的一种结构，便于模型理解可操作对象。

它适合承接“在这个状态执行这个探索动作”的工作。要获得站点级覆盖，外部仍需维护待探索目标、状态标识、恢复路径和跨任务知识。本报告没有把浏览器历史保存等同于网站知识模型。

Crawl4AI：链接发现与内容抓取

bfs_strategy.py 的 `link_discovery()` 会提取内部链接、归一化 URL、检查 visited、应用过滤与评分，然后加入下一层。代码提供深度和页面上限，也包含恢复状态和状态变更回调。

bff_strategy.py 使用优先队列实现按分数选择 URL 的 Best-First 策略。对于文档站、知识门户和内容目录，这些能力可以快速建立第一批页面候选。

这个已核验的深度遍历层仍以 URL/链接为核心。即使浏览器能运行 JavaScript，也不能因此推断系统会自动枚举弹窗、表单和所有业务状态。内容抓取和交互探索可以串联，但应分别验收。

8. 横向对照：该抄哪一层

项目	主要对象	已有产物	最值得借鉴	关键边界
Go-Browse	页面图与探索任务	节点、路径前缀、任务、轨迹	探索主循环和恢复	URL 去重不覆盖全部 UI 状态
Crawljax	交互后 DOM 状态	状态图、DOM、截图、报告	状态判等与转换	还需业务语义整理
AWorld / Recon-Act	辅助执行的经验工具	hint/decision 工具与推理示例	知识转成工具并被调用	示例未核验到完整自动侦察链
Explorer	自生成任务的交互过程	轨迹、HTML、截图、验证信息	任务提出与轨迹整理	未核验到全站 frontier
browser-use	当前任务的浏览器状态	执行历史与结果	页面感知和动作执行	需外加站点探索策略
Crawl4AI	URL 与网页内容	抓取结果、链接、遍历元数据	初始页面发现与批量抓取	不等于业务状态覆盖

以上判断依据前文逐项列出的代码和官方说明。没有用不同模型、不同数据集的论文成功率进行排名：这些数字不能直接反映某个项目作为自动探索组件的优劣。

此外，[hkust-nlp/WebExplorer](#) 虽然名字接近，但官方接口主要是 `search` 和按 URL 的 `browse`，目标是长链路网络问答。它应与 OSU 的 Explorer 区分；若关注站内点击、状态图和业务能力发现，本次不把它列为首选。

9. 如果用于网站知识构建，最小方案是什么

以下是基于上述代码的工程建议，不是任何一个仓库已经完整实现的能力。

输入一个站点、一个明确的登录身份和一个探索预算；输出证据、状态图与经过验证的能力说明。先在一个可重置的测试网站验证这件事，再考虑多站点扩展。

建议的处理顺序是：页面种子 → 待探索队列 → 恢复已知位置 → 观察页面 → 选择未尝试动作 → 执行并记录变化 → 更新状态图 → 提炼有证据的知识。

其中，Go-Browse 提供队列、节点和到达路径的参考；Crawljax 提供状态图思想；browser-use 或现有浏览器工具负责动作。Crawl4AI 可选，用来快速补全链接种子。最后再决定是否将某些稳定能力转成 AWorld 式的工具。

9.1 不要一开始就合并所有状态

建议先同时保留原始 URL、规范化 URL、DOM/可访问性摘要和页面语义标签。去重策略可以迭代，但原始证据丢掉以后很难补回来。

例如，两个不同商品详情页可在页面类型上归为“商品详情”，但不能在原始状态层合成一个节点，因为商品 ID、库存和可选规格可能不同。反过来，同一个列表 URL 的“筛选面板关闭”和“筛选面板展开”应该能被区分。

这里至少有三层对象：浏览器观察状态、页面类型、业务实体。状态图用于导航和恢复；页面类型用于归纳共同能力；业务实体用于描述具体商品、订单或客户。过早把三者混在一个 URL 节点里，会让知识既重复又不可靠。

9.2 产物应该可以逐条追证据

下面的文件名是建议的新产物约定，并非声称现有项目已经输出这些文件：

建议文件	内容
<code>observations.jsonl</code>	每次观察的 URL、身份、DOM/截图引用
<code>transitions.jsonl</code>	源状态、动作、目标状态、执行结果
<code>frontier.json</code>	待探索入口与已尝试动作
<code>site_graph.json</code>	状态、连接与可恢复路径
<code>capabilities.json</code>	能力描述、前提、参数和证据引用
<code>context_model.md</code>	给后续 agent 阅读的精简网站说明

一条有价值的能力记录可以是：“商品列表支持价格升序；在某个分类页执行排序后验证；关联这一次动作前后的截图与 URL；对其他分类是否通用尚未验证。”

这种表达比“该网站支持高级商品搜索”更可用，因为后者没有指出实际做过什么，也没说明适用范围。能力可以先是局部的，验证后再扩大范围。

9.3 恢复状态不能只依赖 `go_back`

浏览器后退只能恢复部分导航历史。打开一个菜单、修改筛选条件、改变购物车内容后，后退并不一定回到相同状态。建议对每条恢复路径设置到达后的检查：页面类型、关键控件和必要数据是否一致。

Go-Browse 的路径前缀是很好的起点，但“记录了路径”和“路径今天仍能重放成功”需要分别统计。重放失败应回到探索队列或标记失效，而不应继续把后续轨迹算成从同一状态出发。

9.4 停止条件要能解释

页面数量上限容易理解，却不够完整。建议同时设置总浏览器动作、总模型调用、重复动作次数，以及连续若干步无新状态时的停止条件。只记录 `max_steps` 会掩盖每一步内部调用多个模型的成本。

覆盖率如果没有完整的网站真值，不宜声称百分之多少。第一阶段可以报告“发现多少不同页面类型、验证多少不同功能、多少路径可重放、多少重复动作”，并固定预算比较增量。

10. 如何做一个可信的小试验

推荐先选一个可重置网站，给固定预算，比较三种探索策略。这是试验设计建议，本次未执行。

组别	机制	想回答的问题
A	同一浏览器 agent，仅用探索提示词	纯模型自主浏览的起点
B	同一 agent，增加队列与路径恢复	结构化调度带来多少增益
C	B，再增加状态判等与重复抑制	是否减少原地循环、发现更多 UI 状态

保持浏览器执行器、模型版本、登录身份、网站初始数据和预算一致。评估不同探索策略时，不同时更换为更强模型或加入特制工具，否则无法解释增益。

观察指标建议包括：独立 URL 数、独立状态数、已验证功能数、重放成功率、重复动作占比、模型调用数、浏览器动作数、token 和时间。还要抽查合并前后的状态，防止通过过细拆分虚增“覆盖”。

之后才评估知识是否帮助执行：固定一组未用于探索目标设计的任务，对比“不注入网站知识”和“注入已冻结知识”。若再加入可执行工具，应单独成组。

离线探索 and 知识整理成本单独列出；在线读取知识的 token 也单独列出。只有报告跨多少任务复用，才能讨论离线成本能否摊薄。一次任务成功不能证明预探索总体更划算。

11. 综合判断与后续走向

最可能有效的路径是组合已有执行器与显式探索状态：模型负责理解页面和提出动作，程序负责记账、去重、预算和恢复。Go-Browse 让这条路线有了可直接阅读的研究实现，Crawljax 提醒我们不能忽视非 URL 状态。

最容易失败的路径是先让模型自由浏览，再把全部轨迹压成一段看似全面的网站说明。它可能记录大量重复观察，却遗漏关键功能；还可能把一次偶然成功概括成通用规则。若没有证据引用和适用前提，后续任务会很难诊断知识为什么出错。

更乐观但需要验证的路径是把稳定的网站知识进一步转成小工具：保留入口条件、参数、结果检查和证据，像 AWorld 的辅助工具那样减少重复推理。这里的瓶颈将从“是否能生成代码”转向“如何证明它可靠、如何发现失效”。

因此，当前最具体的代码阅读任务是：先把 Go-Browse 的 Graph / Node / WebExplore 看通，用 Crawljax 检查状态抽象遗漏，再看 AWorld 怎样消费已整理的网站经验。这样能直接围绕“发现 → 验证 → 保存 → 复用”的实际产物推进。

12. 来源与方法

所有链接访问日期为 2026-09-11。前文文件链接提供逐项证据，以下是主要项目及论文入口：

- [AWorld](#)：核验目录快照、浏览器示例、VisualWebArena 工具代码。
- [Recon-Act 论文](#)：核验侦察/行动框架及实现层级。
- [Go-Browse](#) 与 [论文](#)：核验探索结构及源码入口。
- [Crawljax](#)：核验默认状态类、图与输出构建器。
- [Explorer](#) 与 [论文](#)：核验轨迹生成定位与主流程。
- [browser-use](#)：核验 step 循环和 DOM 服务。
- [Crawl4AI](#) 与 [深度抓取文档](#)：核验 URL 遍历策略。
- [WebExplorer](#)：用于区分网络问答与站内状态探索。

方法上参考 hv-analysis 的纵向路线梳理与横向机制比较，按代码选型问题调整结构。没有进行作者背景、融资或用户口碑调查，也没有把关注度作为技术优劣的证据。本文的工程组合与实验设计为分析建议，所有实际性能结论仍待试跑验证。